SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

CO2 – Scenario-Based Assignment Answers

1. AI-Powered Drone for Emergency Medicine Delivery

i. Greedy Best-First Search: Behaviour, strengths and weaknesses

Greedy Best-First Search selects the node with the smallest heuristic value $h(n)$, which represents the estimated straight-line distance to the destination.

> Behaviour: The drone chooses the path that appears closest to the medicine delivery point.
>
> Strengths: It is fast, easy to implement, and useful when quick decisions are required.
>
> Weaknesses: It ignores the actual travel cost, weather risks, and unexpected obstacles. Therefore, it may select a short but dangerous route.

ii. How A* handles the situation

A* combines the actual cost of reaching a node with the estimated cost of reaching the destination.

$$ f(n)=g(n)+h(n) $$

Where:

> $g(n)$: Actual cost from the starting point to the current node.
>
> $h(n)$: Estimated cost from the current node to the destination.
>
> $f(n)$: Estimated total route cost.

A* selects the path with the lowest estimated total cost. It can consider terrain, weather, fuel consumption, and travel time when calculating $g(n)$. This generally produces better routes than Greedy Best-First Search.

iii. Impact of heuristic accuracy

A good heuristic estimates the remaining distance or cost accurately.

> Greedy Best-First Search: A poor heuristic can lead the drone in the wrong direction because the algorithm depends heavily on $h(n)$.
>
> A:* A poor heuristic can increase the number of nodes explored and slow down the search.

An admissible heuristic that never overestimates the remaining cost helps A* find an optimal path under the standard search assumptions.

iv. Effect of dynamic changes and blocked paths

Blocked roads and changing weather can make previously selected routes unsuitable. The drone must update its map and costs, check whether its current route remains safe, and replan when necessary. Repeated searches may increase computation time, so incremental replanning can improve efficiency.

v. Recommended algorithm

A with real-time route updates and safety constraints* is the most suitable choice. It balances actual travel cost with estimated remaining cost and can account for weather, terrain, and risk. For rapidly changing environments, an incremental replanning algorithm such as D* Lite can be considered. The drone should prioritize safe routes over minimum distance and replan whenever critical conditions change.

2. Smart City Traffic Signal Optimization

Formulation as an optimization problem

The objective is to find signal timings that minimize total waiting time, fuel consumption, and traffic congestion across intersections.

A possible objective function is:

\[J=w_1W+w_2F+w_3C \]

Where:

\(W\) = Total vehicle waiting time.

\(F\) = Total fuel consumption.

\(C\) = Traffic congestion measure.

\(w_1,w_2,w_3\) = Weights representing the importance of each objective.

The constraints include minimum and maximum green-light durations, pedestrian crossing requirements, and safe signal transitions.

Traditional exhaustive search is inefficient because the number of possible signal combinations grows rapidly with the number of intersections. Traffic conditions also change continuously.

i. How Hill Climbing works and its limitations

Hill Climbing starts with an initial set of traffic signal timings. It evaluates nearby alternatives and selects a better solution if one exists. This process continues until no neighboring solution improves the objective.

Limitations:

It may become trapped in a local optimum.

It may fail to explore other promising solutions.

It can struggle when traffic patterns change rapidly.

ii. Local maxima, plateaus, and ridges

Local maximum: A signal configuration improves one intersection but is worse overall than another configuration elsewhere.

Plateau: Several configurations have similar performance, so the algorithm cannot identify a clear improvement.

Ridge: Improvement requires coordinated changes across several intersections, but changing one signal at a time does not appear beneficial.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA): It sometimes accepts a worse solution to escape a local optimum. The probability of accepting worse solutions decreases as the temperature falls.

Genetic Algorithm (GA): It maintains a population of candidate signal plans and improves them through selection, crossover, and mutation. This allows multiple solutions to be explored simultaneously.

Both approaches can explore a larger search space than basic Hill Climbing.

iv. Recommended approach

A Genetic Algorithm combined with real-time traffic data and local signal adjustments is a suitable approach for a large smart city. It explores multiple candidate solutions and can optimize several objectives. Periodic re-optimization helps it adapt to changing traffic patterns. Safety constraints must be enforced, and the algorithm should use a limited computation time to meet real-time requirements.

3. Autonomous Rover Exploring Mars

i. Why an online search agent is required

An online search agent is suitable because the rover explores unknown terrain without having a complete map in advance. It must take actions, observe the results, and update its knowledge continuously.

An offline search agent requires a known environment and a model of possible paths before planning. Since the rover encounters unknown rocks, craters, and terrain conditions, online search is more appropriate.

ii. Challenges of partial observability and dynamic environments

- Partial observability: The rover's sensors cannot detect every obstacle or terrain feature at once.

  Unknown terrain: Some paths may be unsafe or impassable.

  Limited resources: The rover has limited battery power, processing capability, and communication.

  Environmental changes: Newly discovered hazards may require changes to the current route.

  Communication delay: Instructions from Earth may not arrive immediately.

These challenges require the rover to make decisions using incomplete information.

iii. How the rover builds and updates its internal model

The rover uses sensors and cameras to observe its surroundings. It records the location of obstacles, safe paths, and unexplored areas in an internal map.

The process is:

Observe the current environment.

Identify obstacles and possible paths.

Update the internal map using sensor information.

Estimate the cost and safety of possible actions.

Select and execute an action.

Observe the result and update the map again.

This process allows the rover to improve its decisions as it gains information.

iv. Comparison of online, uninformed, and informed search

Search strategy	Description	Limitation
Online search	Explores while acting and updates its knowledge	May encounter unexpected hazards
Uninformed search	Searches without heuristic guidance	May explore many unnecessary states
Informed search	Uses heuristics to guide the search	Depends on heuristic quality and available information

Online search focuses on learning and acting in an unknown environment. Uninformed and informed search can also be used within an online agent when suitable.

v. Most suitable approach

An online search agent combined with heuristic path planning and sensor-based obstacle avoidance is the most suitable approach. It allows the rover to discover unknown terrain, update its map, and change its route when hazards are detected. Safety constraints and conservative movement decisions are essential because the rover cannot rely on a complete map.

4. University Exam Timetable Using CSP

Formulation as a Constraint Satisfaction Problem

A Constraint Satisfaction Problem (CSP) consists of variables, domains, and constraints. The goal is to assign values to all variables without violating any constraint.

i. Variables, domains, and constraints

Variables: Each examination is represented by a variable, such as \(E_1,E_2,E_3\).

Domains: Each examination can be assigned an available time slot and examination hall.

Constraints:

No student overlap: A student must not have two examinations at the same time.

Hall capacity: The number of students assigned to a hall must not exceed its capacity.

Subject ordering: A prerequisite examination must occur before the dependent examination.

Faculty availability: Examinations must be scheduled when the required faculty are available.

Special accommodations: Suitable halls and time slots must be provided for students requiring accommodations.

The objective is to find a valid timetable satisfying all hard constraints.

ii. How backtracking search works

Backtracking assigns a time slot and hall to one examination at a time.

Select an examination that has not been scheduled.

Assign an available time slot and hall.

Check whether the assignment violates any constraint.

If valid, continue with the next examination.

If no valid assignment exists, undo the previous assignment and try another option.

Continue until all examinations are scheduled.

Backtracking guarantees a solution if one exists, provided the search is exhaustive and the constraints are correctly defined.

iii. Constraint propagation using forward checking

Forward checking reduces the domains of unassigned variables after each assignment.

For example, if Mathematics is scheduled at 9:00 AM, other examinations sharing students with Mathematics cannot use that time slot.

The system removes 9:00 AM from the domains of conflicting examinations. If any examination has no remaining valid slot, the algorithm immediately backtracks.

Benefits:

Reduces unnecessary search.

Detects conflicts early.

Improves the efficiency of backtracking.

iv. Real-world challenges and scalability

Large universities may have thousands of students, many courses, limited halls, and faculty availability restrictions. Changes to one examination can affect many others.

A suitable strategy is backtracking with constraint propagation, the Minimum Remaining Values (MRV) heuristic, and degree heuristic.

MRV selects the examination with the fewest available slots.

The degree heuristic prioritizes examinations that constrain the largest number of other examinations.

Forward checking eliminates invalid choices early.

These techniques reduce the search space and help generate feasible timetables more efficiently.

5. Strategic Game AI Using Minimax and Alpha-Beta Pruning

i. How Minimax models the problem

Minimax is a decision-making algorithm used in competitive games involving two opposing players. It assumes that both players choose moves to improve their own outcomes.

MAX player: Selects the move with the highest evaluation value.

MIN player: Selects the move with the lowest evaluation value.

The algorithm builds a game tree, evaluates possible future states, and chooses the move that provides the best outcome against an optimal opponent.

ii. Computational challenges of large game trees

The number of game states increases rapidly with the branching factor and search depth.

If \(b\) is the average number of possible moves and \(d\) is the search depth, the time complexity of Minimax is approximately:

\[O(b^d) \]

For example, when each state has 10 possible moves and the search depth is 5, the tree can contain up to 100,000 leaf nodes.

This makes it difficult to examine every possible move within a strict real-time limit.

iii. How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning improves Minimax by eliminating branches that cannot affect the final decision.

Alpha (α): The best value MAX can guarantee so far.

Beta (β): The best value MIN can guarantee so far.

Pruning occurs when:

$$ \alpha \geq \beta $$

Example: Suppose MAX has already found a move worth 8 points. While examining another move, MIN finds an option worth 5 points. If MIN can force the result to be 5 or lower, MAX will not choose that branch over the existing option worth 8. The remaining unnecessary branches can therefore be skipped.

Alpha-Beta pruning produces the same optimal decision as Minimax when both use the same search depth and evaluation function. With good move ordering, its best-case time complexity is approximately $O(b^{d/2})$.

iv. Design of an evaluation function

A possible evaluation function for a strategy game is:

$$ E(s)=3U+4R+2T+5B-4L $$

Where:

U = Difference in military units between the AI and opponent.

R = Difference in resources.

T = Difference in territory controlled.

B = Difference in strategic objectives captured.

L = Difference in losses suffered.

The coefficients represent the relative importance of each factor. Higher scores indicate more favorable states for the AI. These weights should be tuned and tested for the specific game.

v. Recommended strategy for real-time performance

The most suitable strategy is Minimax with Alpha-Beta pruning, move ordering, and depth-limited search.

Generate the possible moves.

Examine promising moves first.

Apply Alpha-Beta pruning to eliminate unnecessary branches.

Use the evaluation function when the depth limit is reached.

Use iterative deepening to search progressively deeper while time remains.

Return the best move found before the deadline.

This approach reduces computation while preserving optimal decisions within the explored search tree. It balances decision quality, computational efficiency, and real-time responsiveness.

Conclusion

The five scenarios demonstrate how AI search and optimization techniques solve different classes of problems. Greedy Best-First Search and A* support route planning, Hill Climbing and evolutionary optimization improve traffic signals, online search helps autonomous exploration, CSP techniques generate valid examination timetables, and Minimax with Alpha-